前置知识

NoteKing | 图文讲义

Author / Channel	MiniMind / Mokio
Duration	7m17s
Generated by	NoteKing — Video to Illustrated Notes
Date	2026-04-03
Video Link	https://www.bilibili.com/video/BV1T2k6BaEeC
GitHub	github.com/bcefghj/noteking
Xiaohongshu	bcefghj

目录

1 课程概览 2

2 一、PyTorch 张量基础 2

2.1 1.1 什么是张量（Tensor） 2

2.2 1.2 张量的创建与基本操作 3

2.3 1.3 张量的形状（Shape）与索引 3

2.4 1.4 张量的设备管理（CPU/GPU） 4

3 二、矩阵乘法详解 5

3.1 2.1 矩阵乘法的数学原理 5

3.2 2.2 PyTorch 中的矩阵乘法 5

3.3 2.3 逐元素乘法 vs 矩阵乘法 6

3.4 2.4 批量矩阵乘法（Batch MatMul） 7

4 三、注意力机制原理 8

4.1 3.1 为什么需要注意力机制? 8

4.2 3.2 自注意力机制（Self-Attention）核心公式 9

4.3 3.3 查询-键-值（QKV）的直观理解 10

4.4 3.4 缩放点积注意力（Scaled Dot-Product Attention） 10

4.5 3.5 多头注意力机制（Multi-Head Attention） 11

5 四、综合示例：注意力机制的完整实现 13

6 五、本章总结 14

7 关键要点（Key Takeaways） 15

8 思考题 15

9 参考代码 16

第 4 集：前置知识

PyTorch 张量、矩阵乘法与注意力机制原理

1 课程概览

课程名称： MiniMind - PyTorch 从零手敲大模型 **课时：** Episode 4/26 ” 前置知识” (7 分 17 秒) **主要内容：** PyTorch 张量操作、矩阵乘法运算、注意力机制核心原理

---

2 一、PyTorch 张量基础

2.1 1.1 什么是张量（Tensor）

核心概念

张量是 PyTorch 中的核心数据结构，可以简单地理解为**多维数组**。在深度学习中，我们使用张量来存储和操作数据：

- **0 维张量（标量）：** 一个单独的数值，如 `5`
- **1 维张量（向量）：** 一维数组，如 `[1, 2, 3, 4]`
- **2 维张量（矩阵）：** 二维数组，如表格数据
- **3 维及以上：** 高维数组，用于存储图像、文本等复杂数据

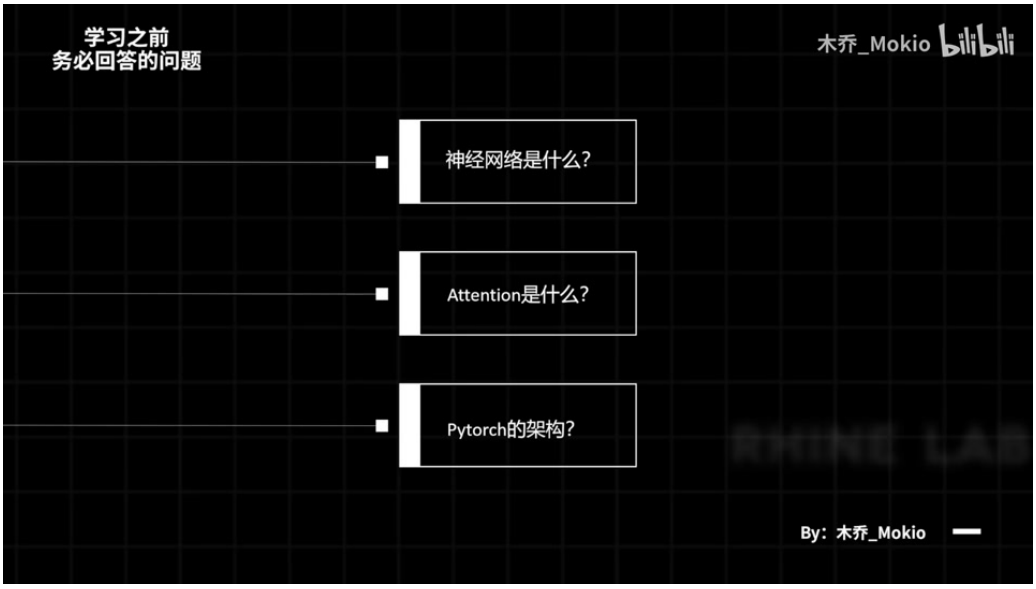

图 1: Video 00:03 [1]

[1] video timestamp 00:03

2.2 1.2 张量的创建与基本操作

```python
import torch

# 创建张量的几种方式
# 方式1: 从Python列表创建
x = torch.tensor([1, 2, 3, 4])
print(f"从列表创建: {x}")

# 方式2: 创建全0或全1张量
zeros = torch.zeros(3, 4)       # 3行4列的全0矩阵
ones = torch.ones(2, 3)         # 2行3列的全1矩阵
print(f"全0张量形状: {zeros.shape}")
print(f"全1张量形状: {ones.shape}")

# 方式3: 创建随机张量
random = torch.randn(3, 3)      # 标准正态分布随机数
print(f"随机张量:\n{random}")
```

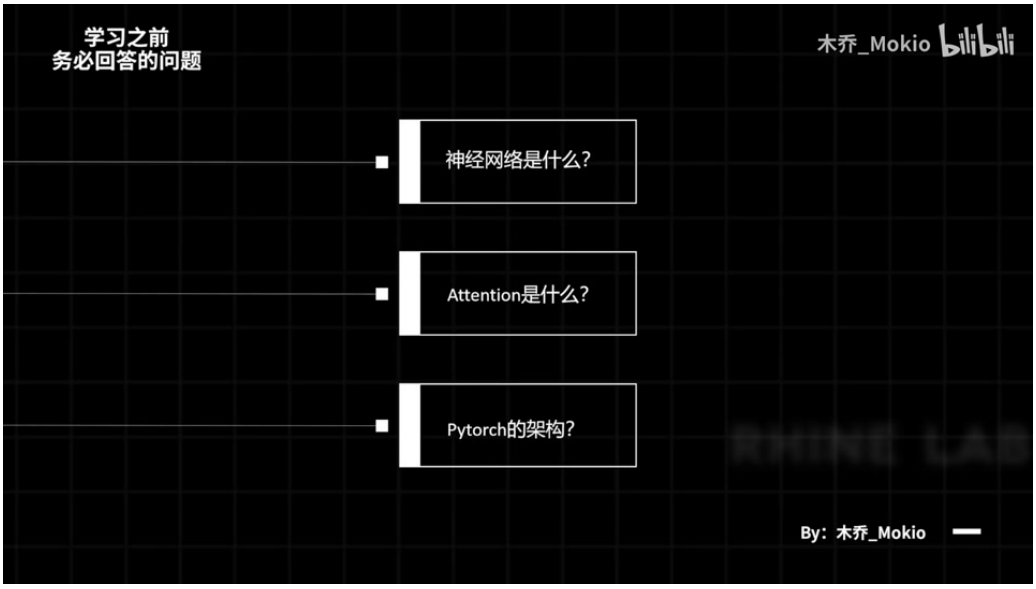

图 2: Video 00:27 [2]

背景知识

在 PyTorch 中，张量默认使用 `torch.float32`（32 位浮点数）数据类型，这在保证计算精度的同时，也确保了较好的计算效率。

2.3 1.3 张量的形状（Shape）与索引

[2] video timestamp 00:27

```python
# 张量的形状属性
x = torch.randn(2, 3, 4)  # 2个3x4的矩阵
print(f"张量形状: {x.shape}")       # torch.Size([2, 3, 4])
print(f"维度数量: {x.dim()}")      # 3

# 张量索引
print(f"第一个矩阵: {x[0]}")
print(f"第一行: {x[0, 0]}")
print(f"特定元素: {x[0, 1, 2]}")
```

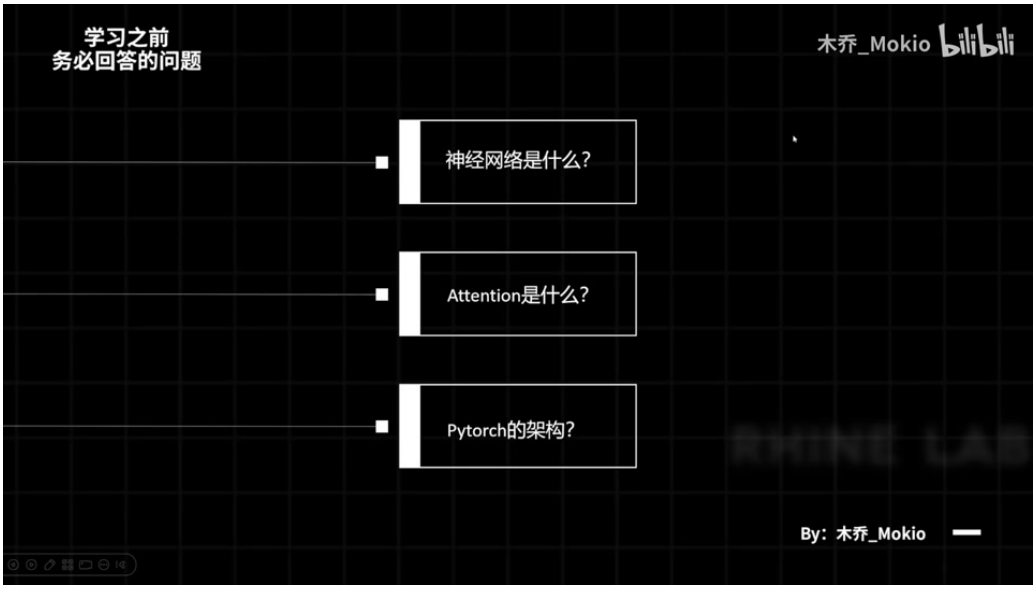

图 3: Video 00:51 [3]

2.4 1.4 张量的设备管理（CPU/GPU）

```python
# 检查CUDA是否可用
print(f"CUDA可用: {torch.cuda.is_available()}")

# 创建张量并移动到GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
x = torch.randn(1000, 1000, device=device)
print(f"张量所在设备: {x.device}")
```

易错点

在编写深度学习代码时，**务必确保模型和张量在同一设备上**（CPU 或 GPU）。设备不匹配会导致运行时错误。

[3] video timestamp 00:51

3 二、矩阵乘法详解

3.1 2.1 矩阵乘法的数学原理

核心概念

对于矩阵 $A \in \mathbb{R}^{m\times n}$ 和矩阵 $B \in \mathbb{R}^{n\times p}$，它们的乘积 $C = AB$ 的定义为：

$$C_{ij} = \sum_{k=1}^{n} A_{ik} \times B_{kj}$$

其中，$C \in \mathbb{R}^{m\times p}$。

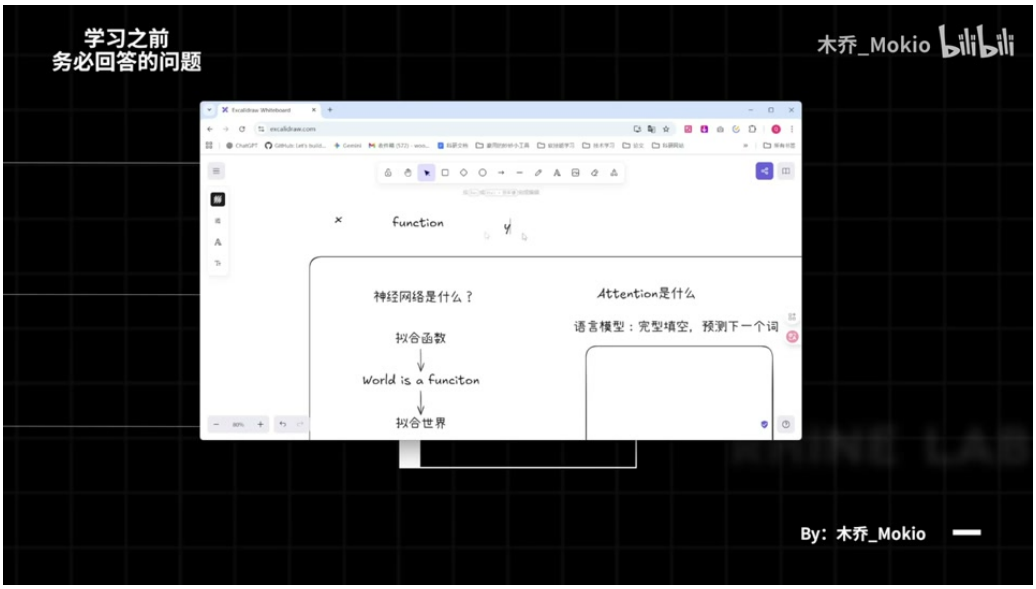

图 4: Video 01:15 [4]

直观理解：结果矩阵 C 的第 i 行第 j 列的元素，等于矩阵 A 的第 i 行与矩阵 B 的第 j 列的**点积**。

3.2 2.2 PyTorch 中的矩阵乘法

```python
# 定义两个矩阵
A = torch.tensor([[1, 2],
                  [3, 4]], dtype=torch.float32)
B = torch.tensor([[5, 6],
                  [7, 8]], dtype=torch.float32)

# 方法1: 使用 @ 运算符
C1 = A @ B
print(f"使用@运算符:\n{C1}")

```

[4] video timestamp 01:15

```python
# 方法2: 使用 torch.matmul
C2 = torch.matmul(A, B)
print(f"使用torch.matmul:\n{C2}")

# 方法3: 使用 torch.mm (仅适用于二维矩阵)
C3 = torch.mm(A, B)
print(f"使用torch.mm:\n{C3}")
```

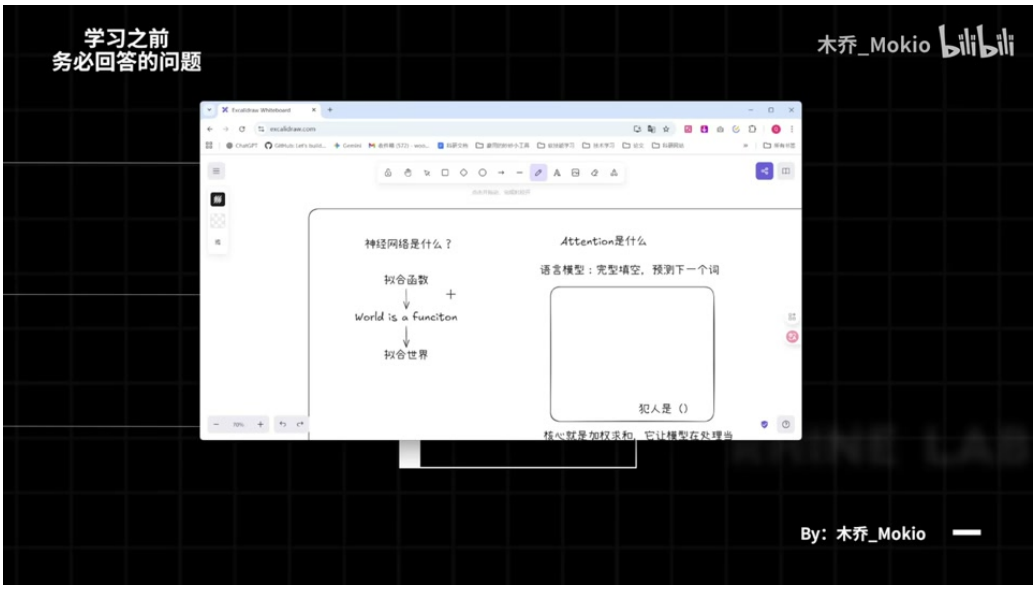

图 5: Video 01:39 [5]

3.3 2.3 逐元素乘法 vs 矩阵乘法

```python
# 逐元素乘法 (Element-wise Multiplication)
# 要求两个矩阵形状完全相同
A = torch.tensor([[1, 2], [3, 4]])
B = torch.tensor([[5, 6], [7, 8]])

element_wise = A * B
print(f"逐元素乘法:\n{element_wise}")

# 矩阵乘法
matmul = torch.matmul(A, B)
print(f"矩阵乘法:\n{matmul}")
```

[5] video timestamp 01:39

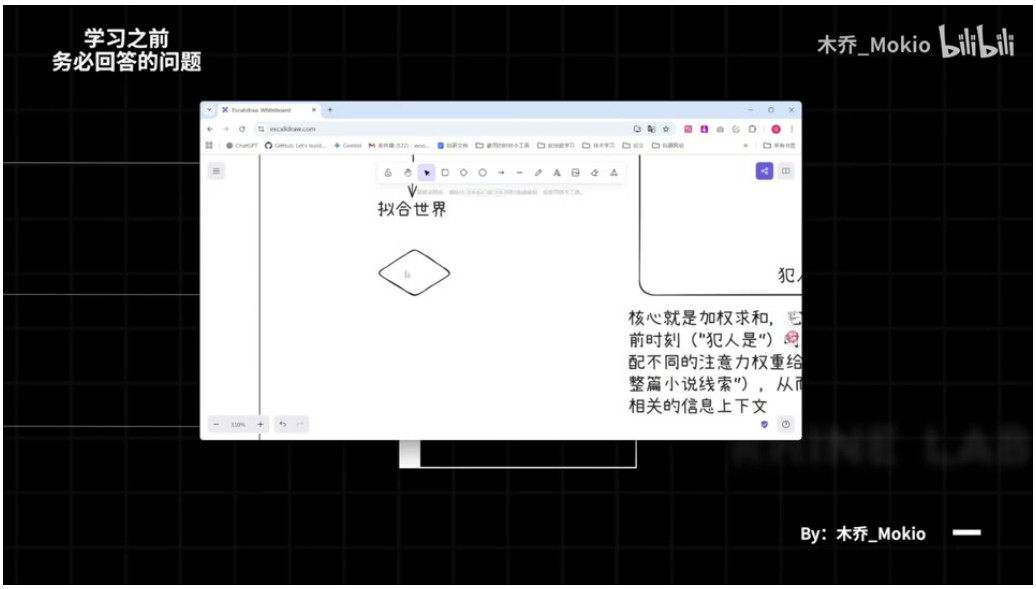

图 6: Video 02:04 [6]

背景知识

- `A * B`：对应位置元素相乘，结果形状不变
- `A @ B`：矩阵乘法，要求第一个矩阵的列数等于第二个矩阵的行数

3.4 2.4 批量矩阵乘法（Batch MatMul）

在深度学习中，我们经常需要对多个矩阵对同时进行乘法运算：

```
# 批量矩阵乘法
# 形状：(batch_size, m, n) @ (batch_size, n, p) -> (batch_size, m, p)
batch_size = 4
m, n, p = 3, 5, 2

A = torch.randn(batch_size, m, n)
B = torch.randn(batch_size, n, p)

# 批量矩阵乘法
C = torch.bmm(A, B)  # bmm: batch matrix multiplication
print(f"批量乘法结果形状：{C.shape}")  # torch.Size([4, 3, 2])
```

[6] video timestamp 02:04

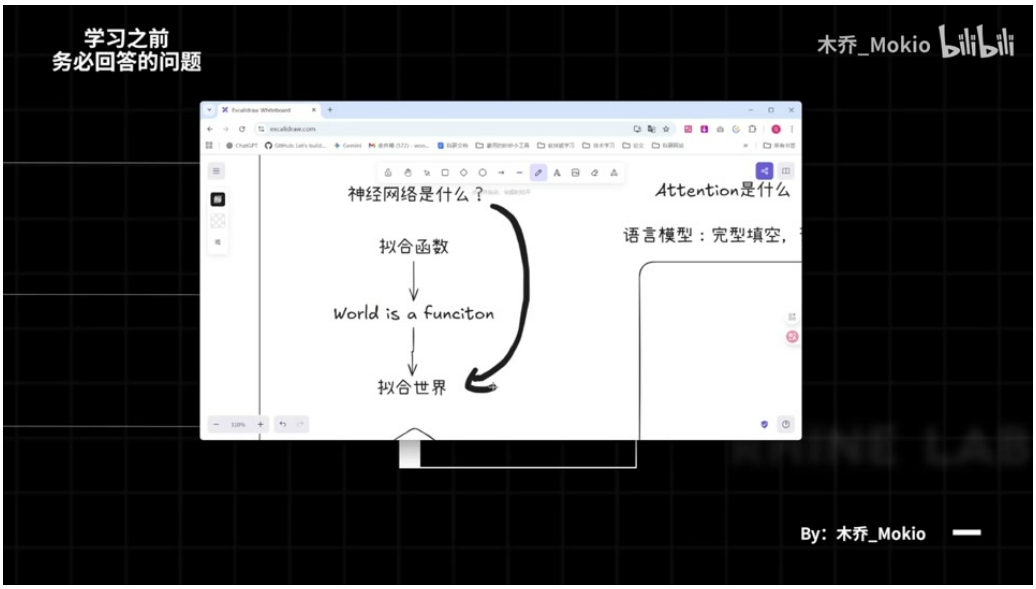

图 7: Video 02:28 [7]

4 三、注意力机制原理

4.1 3.1 为什么需要注意力机制？

核心概念

注意力机制（Attention Mechanism）最早在计算机视觉领域提出，后被广泛应用于自然语言处理。其核心思想是：**让模型学会”关注”输入中最相关的部分**。

传统 RNN 的问题：

- 长距离依赖问题：信息需要经过多个时间步才能传递
- 并行计算受限：必须按顺序处理序列

注意力机制的优势：

- 直接建立任意位置之间的联系
- 支持高效的并行计算
- 可解释性更强

[7] video timestamp 02:28

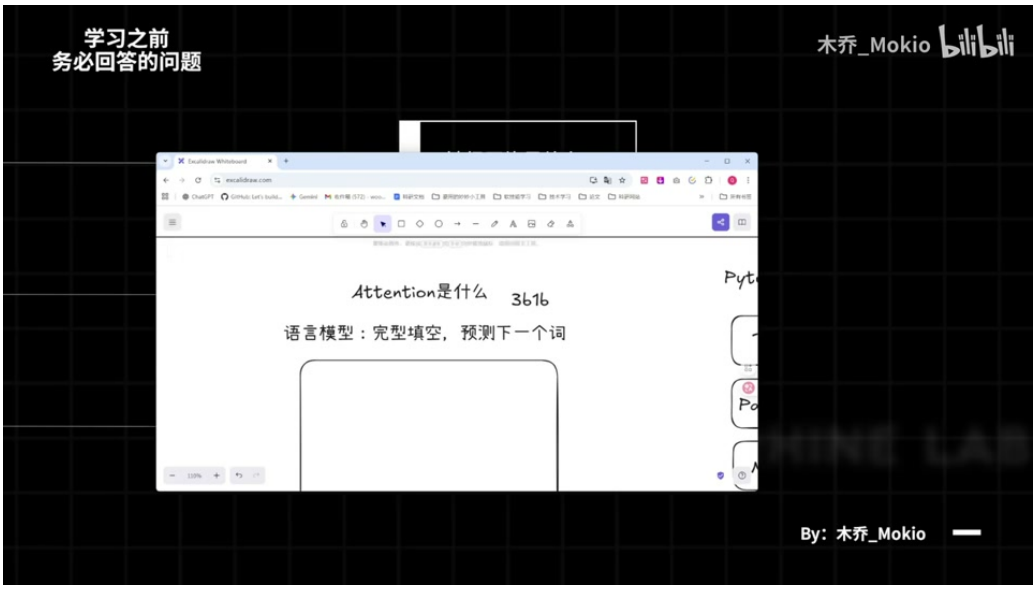

图 8: Video 02:52 [8]

4.2 3.2 自注意力机制（Self-Attention）核心公式

自注意力的核心是计算序列中每个位置对其他位置的”关注程度”：

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

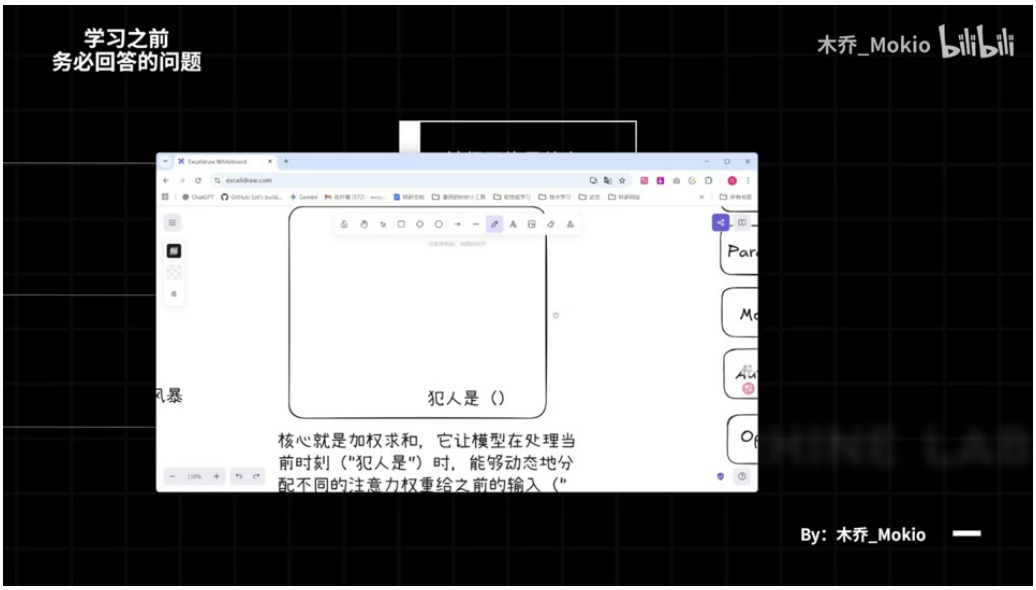

图 9: Video 03:16 [9]

其中：

- **Q（Query）**：查询向量，表示当前位置”在寻找什么”

[8] video timestamp 02:52

[9] video timestamp 03:16

- **K（Key）**：键向量，表示每个位置的”特征标识”
- **V（Value）**：值向量，包含每个位置的”实际信息”
- d_k：键向量的维度，用于缩放

4.3 3.3 查询-键-值（QKV）的直观理解

背景知识

可以将 QKV 机制类比为**图书馆检索系统**：

- **Query（查询）**：你想要查找的书籍主题
- **Key（键）**：每本书的索引标签
- **Value（值）**：书的实际内容

注意力分数 = Query 与所有 Key 的相似度 → 决定应该从哪些 Value 中提取信息。

4.4 3.4 缩放点积注意力（Scaled Dot-Product Attention）

```python
import torch
import torch.nn.functional as F
import math

def scaled_dot_product_attention(Q, K, V, mask=None):
    """
    缩放点积注意力机制实现

    参数:
        Q: 查询张量, shape (batch, num_heads, seq_len, d_k)
        K: 键张量, shape (batch, num_heads, seq_len, d_k)
        V: 值张量, shape (batch, num_heads, seq_len, d_v)
    返回:
        输出张量和注意力权重
    """
    d_k = Q.size(-1)

    # 1. 计算 Q 和 K 的点积
    scores = torch.matmul(Q, K.transpose(-2, -1))

    # 2. 缩放
    scores = scores / math.sqrt(d_k)

    # 3. 应用掩码（可选，用于屏蔽未来位置）
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)
```

```python

    # 4. softmax 归一化
    attention_weights = F.softmax(scores, dim=-1)

    # 5. 与 V 相乘
    output = torch.matmul(attention_weights, V)

    return output, attention_weights
```

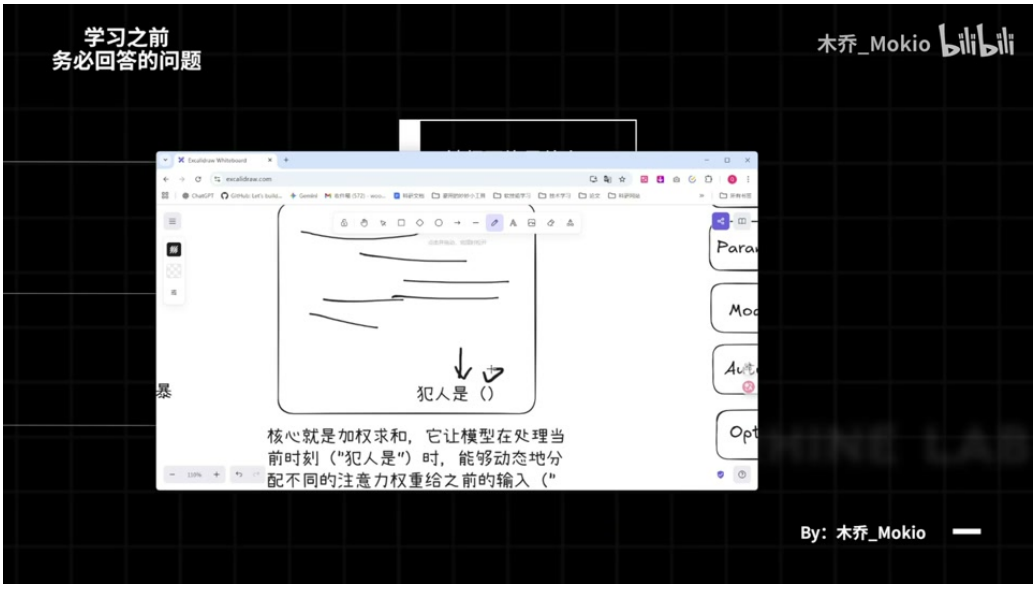

图 10: Video 03:41 [10]

4.5 3.5 多头注意力机制（Multi-Head Attention）

```python
class MultiHeadAttention(torch.nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        self.num_heads = num_heads
        self.d_model = d_model
        self.d_k = d_model // num_heads

        # 线性变换层
        self.W_Q = torch.nn.Linear(d_model, d_model)
        self.W_K = torch.nn.Linear(d_model, d_model)
        self.W_V = torch.nn.Linear(d_model, d_model)
        self.W_O = torch.nn.Linear(d_model, d_model)

    def forward(self, Q, K, V, mask=None):
        batch_size = Q.size(0)
```

[10] video timestamp 03:41

```python

        # 线性变换并分头
        Q = self.W_Q(Q).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
        K = self.W_K(K).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
        V = self.W_V(V).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)

        # 计算注意力
        x, attn_weights = scaled_dot_product_attention(Q, K, V, mask)

        # 合并多头并通过输出层
        x = x.transpose(1, 2).contiguous().view(batch_size, -1, self.d_model)
        output = self.W_O(x)

        return output, attn_weights
```

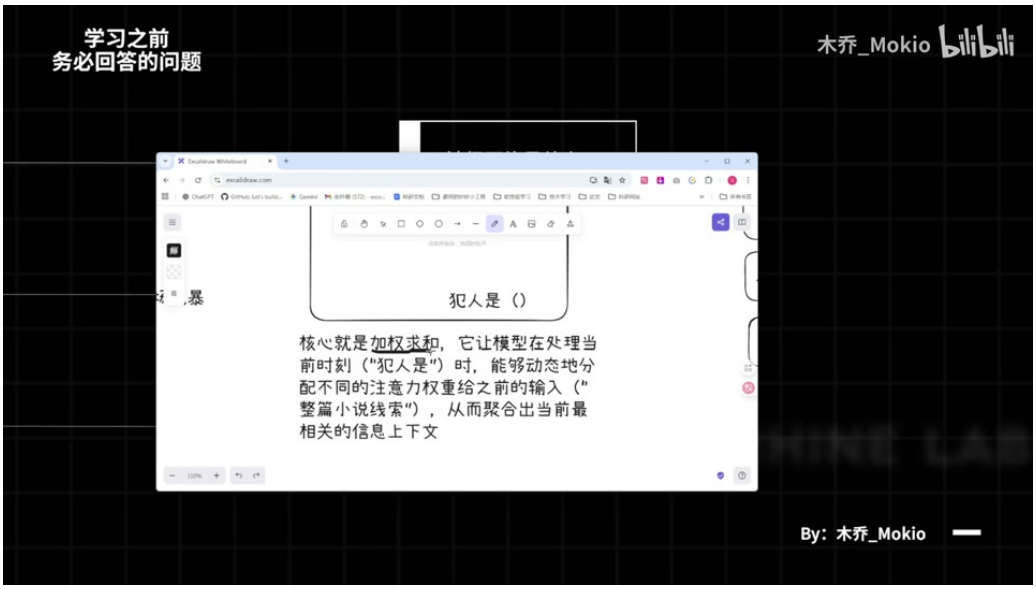

图 11: Video 04:05 [11]

背景知识

多头注意力的意义：

- 每个"头"可以关注不同的语义特征
- 有的头可能关注语法结构
- 有的头可能关注语义相关性

[11] video timestamp 04:05

- 多个头并行学习，最后合并，捕获更丰富的信息

---

5 四、综合示例：注意力机制的完整实现

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class SelfAttention(nn.Module):
    """简化的自注意力层"""

    def __init__(self, embed_size, heads):
        super(SelfAttention, self).__init__()
        self.embed_size = embed_size
        self.heads = heads
        self.head_dim = embed_size // heads

        assert self.head_dim * heads == embed_size, "embed_size 必须能被 heads 整除"

        # QKV 变换矩阵
        self.values = nn.Linear(embed_size, embed_size)
        self.keys = nn.Linear(embed_size, embed_size)
        self.queries = nn.Linear(embed_size, embed_size)
        self.fc_out = nn.Linear(embed_size, embed_size)

    def forward(self, values, keys, query, mask):
        N = query.shape[0]
        value_len, key_len, query_len = values.shape[1], keys.shape[1], query.shape[1]

        # 线性变换
        values = self.values(values)
        keys = self.keys(keys)
        queries = self.queries(query)

        # reshape: (N, seq_len, embed_size) -> (N, seq_len, heads, head_dim)
        values = values.reshape(N, value_len, self.heads, self.head_dim)
        keys = keys.reshape(N, key_len, self.heads, self.head_dim)
        queries = queries.reshape(N, query_len, self.heads, self.head_dim)

```

```python
        # 计算注意力分数
        energy = torch.einsum("nqhd,nkhd->nhqk", [queries, keys])
        # einsum解释: (N, query_len, heads, head_dim) @ (N, key_len, heads, head_dim)
        #             -> (N, heads, query_len, key_len)

        if mask is not None:
            energy = energy.masked_fill(mask == 0, float("-1e20"))

        attention = F.softmax(energy / math.sqrt(self.head_dim), dim=3)

        # 加权求和
        out = torch.einsum("nhql,nlhd->nqhd", [attention, values]).reshape(
            N, query_len, self.heads * self.head_dim
        )

        return self.fc_out(out)

# 使用示例
embed_size = 256
heads = 8
seq_length = 100
batch_size = 32

attention = SelfAttention(embed_size, heads)
x = torch.randn(batch_size, seq_length, embed_size)

output = attention(x, x, x, mask=None)
print(f"输入形状: {x.shape}")
print(f"输出形状: {output.shape}")
```

---

6 五、本章总结

核心知识点

1. **PyTorch 张量**

- 张量是多维数组，是深度学习中的基本数据结构
- 支持 CPU/GPU 切换，需注意设备匹配

1. **矩阵乘法**

- `@` 或 `torch.matmul()` 用于矩阵乘法
- `*` 用于逐元素乘法
- `torch.bmm()` 用于批量矩阵乘法

1. **注意力机制**

- 核心公式：$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$
- QKV 分离让模型学习不同的语义表示
- 多头注意力捕获多种特征关系

---

7 关键要点（Key Takeaways）

概念	核心要点
张量	多维数组，支持 GPU 加速，自动求导
矩阵乘法	使用 BACKSLASH_PLACEHOLDERtexttt{@} 或 BACKSLASH_PLACEHOLDERtexttt
注意力机制	QKV 分离 → 点积相似度 → softmax 归一化 → 加权求和
缩放因子	$BACKSLASH_PLACEHOLDERsqrt\{d_k\}$ 防止点积过大导致梯度消失

---

8 思考题

1. **问题一：** 在注意力机制中，为什么要对 QK^T 的结果除以 $\sqrt{d_k}$?

提示：*考虑 softmax 函数的输入值范围对梯度的影响。*

1. **问题二：** 如果我们有形状为（`batch_size`，`seq_len`，`d_model`）的输入张量，想要实现自注意力，需要如何设计 Q、K、V 的变换矩阵?

提示：*自注意力中 Q、K、V 都来自同一个输入。*

---

9 参考代码

```python
# 完整的自注意力计算流程
import torch
import torch.nn.functional as F
import math

def self_attention(X, d_k):
    """
    最简单的自注意力实现

    Args:
        X: 输入张量 (batch, seq_len, d_model)
        d_k: key/query 维度
    """
    batch, seq_len, d_model = X.shape

    # 简单的线性变换得到 Q, K, V
    W_q = torch.randn(d_model, d_k)
    W_k = torch.randn(d_model, d_k)
    W_v = torch.randn(d_model, d_model)

    Q = X @ W_q  # (batch, seq_len, d_k)
    K = X @ W_k  # (batch, seq_len, d_k)
    V = X @ W_v  # (batch, seq_len, d_model)

    # 计算注意力分数
    scores = Q @ K.transpose(-2, -1) / math.sqrt(d_k)  # (batch, seq_len,
        seq_len)
    attention_weights = F.softmax(scores, dim=-1)

    # 加权求和
    output = attention_weights @ V  # (batch, seq_len, d_model)

    return output, attention_weights
```

下一节预告：我们将学习 Transformer 架构的完整实现，理解如何堆叠多头注意力层和前馈网络构建完整的编码器/解码器结构。